



Effects for Fetching Data



Learning Objectives

- Explain the method signature of React's effect hook
- Build a component with an effect hook that requests data from an API only on component mounting
- Build a component that has a state and an effect hook



Intro

So far we've been working with local data

However, we should be able to fetch data from a server, just like we did with plain JavaScript

Q: Looking at this component though, where would you write your fetch?

```
function App () {  
  const [todos, setTodos] = useState([])  
  
  return <div className="app">  
    {todos.map(todo =>  
      <li key={todo.id}>{todo.title}</li>  
    )}  
  </div>  
}
```



Intro

Q: How about here? What do you expect this code to do?

```
function App () {  
  const [todos, setTodos] = useState([])  
  
  fetch("https://jsonplaceholder.typicode.com/todos")  
    .then(res => res.json())  
    .then(setTodos)  
  
  return <div className="app">  
    {todos.map(todo =>  
      <li key={todo.id}>{todo.title}</li>  
    )}  
  </div>  
}
```



Time for some live coding

There's only one way to find out!



Rerendering forever

So, turns out this code is making an **infinite** number of fetches because:

- Calling **setTodos** updates the state
- Updating state **re-renders** the component
- Re-rendering the component makes the fetch request again, which **calls setTodos again**





useEffect

So, what we really need is a way to send a request **only once**, when the component first renders (sometimes referred to as **mounting or initialising**)

Any code that is not directly related to state and props in React can be thought of as an **effect**, and to handle these, React gives us another useful built-in hook: **useEffect**



Time for some live coding

Let's check it out!



useEffect

So, this is the code that we added to achieve the result we wanted

Let's explore it in more detail!

Q: What's the signature of useEffect?

```
function App () {  
  // ...  
  useEffect(() => {  
    getTodos().then(setTodos)  
  }, [])  
  // ...  
}
```



useEffect signature

useEffect takes two arguments:

- A **function** to run
- An **array of dependencies**

```
function App () {  
  // ...  
  useEffect(() => {  
    fetch("/todos")  
      .then(res => res.json())  
      .then(setTodos)  
  }, [])  
  // ...  
}
```



What useEffect does

By default, an effect will run **after every render**, but we can control this by adding its **second argument**: an array of dependencies

Every time the component **re-renders**, React will check if any of those dependencies has **changed**, if it has, it will **rerun** the effect, otherwise it will **skip it**

Q: When will this effect run?

```
function App () {  
  // ...  
  useEffect(() => {  
    fetch("/todos")  
      .then(res => res.json())  
      .then(setTodos)  
  }, [])  
  // ...  
}
```



useEffect's second argument

Adding an **empty array** as a second argument ensures this code only runs the first time the component renders, and **never again**

```
function App () {  
  // ...  
  
  useEffect(() => {  
    getTodos().then(setTodos)  
  }, [])  
  
  // ...  
}
```



Time for some live coding

Open a codesandbox and render a list of data from one of these APIs [here](#)



Time for some practice

Pick an API from this [list](#) and render a list of data on the page.

Here are some curated suggested to get working quickly

- **A Random Fun Quotes (subjectively...)**
 - <https://breakingbadquotes.xyz/>
- **The Weather**
 - <https://openweathermap.org/api> (needs an API key)
- - ???



Time for some practice cont.

Alternatively, create a data.json file with the following contents and serve it using json-server:

```
{
  "todos": [
    {
      "id": 1, "title": "task 1", "done": false
    },
    {
      "id": 2, "title": "task 2", "done": false
    },
    {
      "id": 3, "title": "task 3", "done": true
    }
  ]
}
```